

Desarrollo de Clase

Post-Quantum Cryptography

Resumen técnico desarrollado a partir de la presentación de clase enviada
Universidad Nacional de Ingeniería

Tema central: entender por qué la computación cuántica amenaza a RSA, Diffie-Hellman, ECDH, DSA y ECDSA, y cómo la criptografía post-cuántica propone nuevos mecanismos como ML-KEM, ML-DSA y SLH-DSA.

Tabla de contenidos

1. Idea central de la clase
2. Funciones one-way y trapdoor
3. Criptografía de clave pública
4. RSA y el problema de factorización
5. Algoritmos clásicos amenazados
6. Algoritmo de Shor
7. Por qué nace la criptografía post-cuántica
8. Harvest Now, Decrypt Later
9. Ejemplo con TLS 1.3
10. Estándares NIST PQC
11. Significado de los nombres
12. Diferencia entre Key Exchange y Key Encapsulation
13. Librerías PQC disponibles
14. Resumen técnico y conclusión

1. Idea central de la clase

La clase desarrolla el tema Post-Quantum Cryptography, es decir, criptografía diseñada para resistir ataques de computadoras cuánticas. El punto de partida es que gran parte de la criptografía moderna depende de problemas matemáticos que son difíciles para una computadora clásica, pero que podrían resolverse eficientemente con una computadora cuántica suficientemente grande.

El problema principal es el siguiente: RSA, Diffie-Hellman, ECDH, DSA y ECDSA son seguros actualmente porque una computadora clásica no puede resolver de forma eficiente ciertos problemas matemáticos. Sin embargo, el algoritmo de Shor puede resolver esos problemas en tiempo polinomial usando computación cuántica.

Por esta razón aparece la criptografía post-cuántica, abreviada como PQC. Su objetivo es construir algoritmos que sigan siendo seguros incluso frente a atacantes con capacidad cuántica.

2. Funciones one-way y trapdoor

2.1 Función one-way

Una función one-way es una función fácil de calcular, pero difícil de invertir. Esta idea es fundamental en criptografía porque permite transformar información de manera eficiente en una dirección, pero vuelve computacionalmente inviable recuperar la entrada original a partir de la salida.

$$f : D \rightarrow R$$

Dado un valor x , calcular $y = f(x)$ debe ser eficiente. Sin embargo, dado $y = f(x)$, encontrar x debe ser computacionalmente difícil.

$$x \rightarrow f(x) \text{ es fácil } f(x) \rightarrow x \text{ es difícil}$$

En criptografía, esta propiedad permite construir esquemas donde se puede cifrar o verificar de forma pública, pero no se puede invertir la operación sin información secreta.

2.2 Trapdoor function

Una trapdoor function es una función one-way con una puerta secreta. Sin la información secreta, invertir la función es difícil. Con la información secreta, la inversión se vuelve eficiente.

$$y = f(x) \quad x = f^{-1}(y, t)$$

Aquí t representa la información secreta o trapdoor. Esta idea explica por qué la criptografía de clave pública puede permitir cifrado público y descifrado privado, o verificación pública y firma privada.

3. Criptografía de clave pública

La criptografía de clave pública usa un par de claves: una clave pública y una clave privada.

$$(pk, sk) \quad pk = \text{clave pública} \quad sk = \text{clave secreta o privada}$$

La clave pública puede compartirse con cualquier persona. La clave privada debe mantenerse secreta. Los dos usos principales son cifrado y firma digital.

3.1 Cifrado

En un esquema de cifrado de clave pública, una persona usa la clave pública del receptor para cifrar un mensaje. Luego, solamente quien posee la clave privada puede descifrarlo.

$$c = \text{Enc}(pk, m) \quad m = \text{Dec}(sk, c)$$

Donde m es el mensaje original y c es el texto cifrado.

3.2 Firma digital

En una firma digital, el dueño de la clave privada firma un mensaje. Luego, cualquier persona puede verificar la firma usando la clave pública.

$$\text{sigma} = \text{Sign}(sk, m) \quad \text{Verify}(pk, m, \text{sigma}) \in \{0,1\}$$

Si el resultado de la verificación es 1, la firma es válida. Si el resultado es 0, la firma se rechaza.

4. RSA y el problema de factorización

RSA se basa en la dificultad de factorizar enteros grandes. La idea matemática central es que multiplicar dos primos grandes es fácil, pero recuperar esos primos a partir del producto es difícil para una computadora clásica.

- Se eligen dos primos grandes p y q .
- Se calcula $n = p \cdot q$.
- Se publica n .
- Se mantienen secretos p y q .

$$p * q = n$$

La seguridad depende de que sea difícil recuperar p y q conociendo solamente n . Para números de gran tamaño, por ejemplo módulos RSA de 2048 bits, no existe actualmente un algoritmo clásico eficiente que permita factorizar de forma práctica.

4.1 RSA explicado con claves

En RSA se calcula la función phi de Euler del módulo n :

$$\phi(n) = (p - 1)(q - 1)$$

Luego se escoge un exponente público e tal que sea coprimo con $\phi(n)$:

$$\text{gcd}(e, \phi(n)) = 1$$

Después se calcula el exponente privado d , que satisface:

$$e \cdot d \equiv 1 \pmod{\phi(n)}$$

La clave pública y la clave privada se representan como:

$$pk = (n, e) \quad sk = d$$

Para cifrar un mensaje m se calcula:

$$c \equiv m^e \pmod{n}$$

Para descifrar se calcula:

$$m \equiv c^d \pmod{n}$$

El punto importante es que obtener d sin conocer p y q es computacionalmente difícil. Por eso, romper RSA equivale en la práctica a factorizar n .

5. Algoritmos clásicos amenazados

La presentación indica que gran parte de la infraestructura criptográfica mundial se basa en algoritmos como RSA, Diffie-Hellman, ECDH, DSA y ECDSA. Estos algoritmos comparten una característica: su seguridad depende de problemas matemáticos considerados difíciles para computadoras clásicas.

5.1 Factorización de enteros

La factorización de enteros es la base de RSA. El problema consiste en encontrar p y q a partir de n cuando $n = p \cdot q$.

$n = p \cdot q$ Conocido n , encontrar p y q es difícil.

5.2 Logaritmo discreto

El logaritmo discreto es la base de Diffie-Hellman, DSA, ECDH y ECDSA. Dado un valor de la forma $g^a \bmod p$, encontrar el exponente a es difícil.

$g^a \bmod p \rightarrow$ encontrar a

Ese valor a es el logaritmo discreto. La dificultad de obtenerlo es lo que protege muchos protocolos de intercambio de claves y firmas digitales.

6. Algoritmo de Shor

El punto crítico de la clase es el algoritmo de Shor. En 1994, Peter Shor propuso un algoritmo cuántico capaz de resolver en tiempo polinomial dos problemas fundamentales: la factorización de enteros y el logaritmo discreto.

- Factorización de números enteros grandes.
- Logaritmo discreto.

Estos dos problemas son la base de la seguridad de RSA, Diffie-Hellman, ECDH, DSA y ECDSA. Por tanto, una computadora cuántica suficientemente grande y estable podría romper los supuestos de seguridad de estos algoritmos.

6.1 Comparación clásica y cuántica

En computación clásica, factorizar un número grande de 2048 bits requiere un tiempo extremadamente alto. En computación cuántica, Shor reduce el problema a complejidad polinomial.

Esto no significa que cualquier computadora cuántica actual pueda romper RSA inmediatamente. Significa que, cuando exista una computadora cuántica suficientemente grande, los algoritmos clásicos basados en factorización y logaritmo discreto dejarán de ser seguros.

7. Por qué nace la criptografía post-cuántica

La criptografía post-cuántica busca reemplazar los algoritmos vulnerables a Shor por nuevos algoritmos cuya seguridad se base en problemas matemáticos para los que no se conocen algoritmos cuánticos eficientes.

Las principales familias mencionadas en la clase son:

- Lattices o retículos.
- Codes o códigos correctores de errores.
- Hashes o funciones hash.
- Isogenies o isogenias.
- Multivariate o sistemas multivariados.

No todas estas familias han tenido el mismo éxito. Algunas propuestas fueron descartadas por ataques posteriores. Por eso es necesario un proceso formal de evaluación, selección y estandarización.

8. Harvest Now, Decrypt Later

Una idea importante de la clase es el ataque Harvest Now, Decrypt Later, que en español puede entenderse como capturar ahora y descifrar después.

En este escenario, un atacante intercepta y almacena tráfico cifrado hoy, aunque todavía no pueda descifrarlo. En el futuro, cuando disponga de una computadora cuántica suficientemente poderosa, podría usarla para romper el intercambio de claves y descifrar la información almacenada.

Este riesgo es especialmente serio para información que debe permanecer secreta durante muchos años.

- Datos militares.
- Datos médicos.
- Información financiera.
- Secretos industriales.
- Comunicaciones gubernamentales.

Por eso la migración a criptografía post-cuántica no debe empezar recién cuando exista una computadora cuántica capaz de romper RSA o ECDH. La transición debe empezar antes.

9. Ejemplo con TLS 1.3

La clase usa TLS como ejemplo real. En una sesión típica de TLS 1.3, cliente y servidor realizan un intercambio de claves usando ECDHE. El objetivo es que ambos obtengan una clave de sesión simétrica para cifrar el tráfico con un algoritmo como AES-256-GCM.

- El cliente envía una clave pública efímera g^a .
- El servidor envía una clave pública efímera g^b .
- Ambos calculan el secreto compartido g^{ab} .

- Con ese secreto derivan una clave simétrica de sesión.

Cliente: g^a Servidor: g^b Secreto compartido: g^{ab}

Un atacante que observa la red ve g^a y g^b , pero no conoce a ni b. En computación clásica, no puede obtener a partir de g^a porque tendría que resolver el logaritmo discreto.

9.1 Amenaza cuántica sobre TLS

En el futuro, un atacante cuántico podría usar Shor para recuperar a desde g^a . Luego calcularía g^{ab} , recuperaría la clave de sesión y descifraría el tráfico almacenado.

captura g^a , g^b , c espera una computadora cuántica usa Shor para obtener a calcula g^{ab} recupera la clave de sesión descifra el tráfico

Este ejemplo explica por qué el problema Harvest Now, Decrypt Later es urgente. El atacante no necesita romper el sistema hoy; solo necesita capturar la información y esperar.

10. Estándares NIST PQC

La presentación muestra los estándares y convenciones de nombres de NIST para criptografía post-cuántica. Los principales algoritmos se organizan según su propósito: encapsulamiento de clave o firma digital.

FIPS	Nombre NIST	Nombre original	Propósito	Familia
FIPS 203	ML-KEM	CRYSTALS-Kyber	Key Encapsulation	Lattice (Module-LWE)
FIPS 204	ML-DSA	CRYSTALS-Dilithium	Firma digital	Lattice (Module-LWE/SIS)
FIPS 205	SLH-DSA	SPHINCS+	Firma digital	Hash-based
FIPS 206 draft	FN-DSA	Falcon	Firma digital	Lattice (NTRU)
Separado	HQC	HQC	Backup KEM	Code-based

En esta clasificación, ML-KEM se usa para encapsular claves, mientras que ML-DSA y SLH-DSA se usan para firmas digitales.

11. Significado de los nombres

11.1 ML

ML significa Module-Lattice. Indica que el algoritmo está basado en retículos modulares. Por ejemplo, ML-KEM significa Module-Lattice Based Key Encapsulation Mechanism.

ML = Module-Lattice ML-KEM = Module-Lattice Based Key Encapsulation Mechanism

11.2 SLH

SLH significa Stateless Hash-Based. Se usa para firmas digitales basadas en funciones hash sin estado.

SLH-DSA = Stateless Hash-Based Digital Signature Algorithm

11.3 FN

FN significa Fourier-NTRU. Está asociado a Falcon y a técnicas basadas en NTRU y muestreo mediante FFT.

11.4 KEM

KEM significa Key Encapsulation Mechanism. No es exactamente un intercambio de claves clásico, sino un mecanismo para encapsular una clave de sesión.

11.5 DSA

DSA significa Digital Signature Algorithm. Se usa para firmas digitales.

12. Diferencia entre Key Exchange y Key Encapsulation

12.1 Diffie-Hellman clásico

En Diffie-Hellman clásico, Alice y Bob publican valores relacionados con exponentes secretos. Luego ambos calculan el mismo secreto compartido.

Alice publica: g^a Bob publica: g^b Alice calcula: $(g^b)^a = g^{ab}$ Bob calcula: $(g^a)^b = g^{ab}$

El intercambio es simétrico: ambos participantes contribuyen de manera equivalente y llegan al mismo secreto gracias a la estructura algebraica del grupo.

12.2 KEM post-cuántico

En un KEM post-cuántico, la estructura cambia. Alice genera una clave pública y una clave secreta. Bob genera o deriva una clave de sesión y la encapsula usando la clave pública de Alice. Alice recupera la misma clave usando su clave secreta.

KeyGen() $\rightarrow$ (pk, sk) Encaps(pk) $\rightarrow$ (c, K) Decaps(sk, c) $\rightarrow$ K

Aquí c es el ciphertext o encapsulado, y K es la clave compartida final. Esta es la estructura típica de algoritmos como ML-KEM, antes conocido como Kyber.

12.3 Por qué se habla de encapsulación

En esquemas basados en retículos no siempre existe una operación conmutativa tan limpia como en Diffie-Hellman, donde $g^{ab} = g^{ba}$. Por eso el enfoque natural es encapsular una clave aleatoria de sesión, no construirla mediante una operación de grupo conmutativa.

Diffie-Hellman: $g^{ab} = g^{ba}$ PQC KEM: encapsular una clave aleatoria de sesión

13. Librerías PQC disponibles

13.1 Open Quantum Safe

Open Quantum Safe, abreviado OQS, es un proyecto académico-industrial que implementa algoritmos post-cuánticos. Su objetivo es facilitar pruebas, investigación y migración hacia algoritmos resistentes a ataques cuánticos.

- Implementa estándares NIST.

- Integra con OpenSSL.
- Soporta ML-KEM y ML-DSA.
- Es útil para investigación y pruebas de interoperabilidad.

13.2 liboqs

liboqs es la librería base del ecosistema Open Quantum Safe. Ofrece implementaciones de algoritmos PQC y bindings para varios lenguajes.

- C.
- C++.
- Python.
- Go.
- Rust.
- Java mediante bindings.

Su uso recomendado es investigación, prototipos y laboratorios de migración.

13.3 OpenSSL 3 + OQS Provider

OpenSSL 3 con OQS Provider permite experimentar con criptografía post-cuántica o híbrida dentro de OpenSSL. Esto es útil para probar TLS híbrido, certificados PQC y escenarios de migración.

Un modo híbrido combina un algoritmo clásico y uno post-cuántico. Por ejemplo:

ECDHE + ML-KEM

Así, si uno de los componentes falla, el otro todavía puede contribuir a proteger la sesión.

13.4 Bouncy Castle PQC

Bouncy Castle PQC es especialmente importante para Java. Permite trabajar con algoritmos post-cuánticos dentro del ecosistema Java y se integra con JCA/JCE.

- ML-KEM, antes Kyber.
- ML-DSA, antes Dilithium.
- FN-DSA, asociado a Falcon.
- SPHINCS+.

Esto es útil para proyectos Java de firmas digitales, PKI, certificados y pruebas de migración criptográfica.

14. Resumen técnico y conclusión

La clase puede resumirse de la siguiente manera: la criptografía clásica de clave pública funciona porque ciertos problemas matemáticos son difíciles para computadoras clásicas. RSA depende de la factorización de enteros. Diffie-Hellman, DSA, ECDH y ECDSA dependen del logaritmo discreto.

El algoritmo de Shor rompe ambos supuestos en el modelo cuántico, porque permite resolver factorización y logaritmo discreto en tiempo polinomial. Por eso se necesitan nuevos

algoritmos post-cuánticos.

NIST estandarizó algoritmos como ML-KEM, ML-DSA y SLH-DSA. ML-KEM se usa para encapsulamiento de claves. ML-DSA y SLH-DSA se usan para firmas digitales. La migración es urgente por el riesgo de Harvest Now, Decrypt Later.

14.1 Esquema mental de la clase

Funciones one-way -> Trapdoor functions -> Criptografía de clave pública -> RSA, DH, ECDH, DSA, ECDSA -> Factorización y logaritmo discreto -> Algoritmo de Shor -> Amenaza cuántica -> Post-Quantum Cryptography -> NIST PQC Standards -> ML-KEM, ML-DSA, SLH-DSA -> Implementación con OQS, OpenSSL y Bouncy Castle

14.2 Conclusión para exponer

La seguridad criptográfica actual no se rompe porque los algoritmos estén mal diseñados, sino porque su seguridad depende de problemas matemáticos que son difíciles bajo el modelo clásico de computación. Con la aparición de la computación cuántica, especialmente por el algoritmo de Shor, los problemas de factorización y logaritmo discreto dejan de ser una base segura para el futuro.

La criptografía post-cuántica propone nuevos algoritmos basados en retículos, códigos y funciones hash. Los estándares actuales más importantes son ML-KEM para encapsulamiento de claves, y ML-DSA y SLH-DSA para firmas digitales.

El mensaje final de la clase es que la migración debe empezar desde ahora. Un atacante puede capturar tráfico cifrado en el presente y descifrarlo en el futuro cuando disponga de capacidad cuántica suficiente.

15. Glosario breve

Término	Significado
PQC	Post-Quantum Cryptography. Criptografía resistente a ataques cuánticos.
KEM	Key Encapsulation Mechanism. Mecanismo para encapsular una clave de sesión.
ML-KEM	Algoritmo NIST para encapsulamiento de claves basado en retículos modulares.
ML-DSA	Algoritmo NIST de firma digital basado en retículos modulares.
SLH-DSA	Algoritmo NIST de firma digital basado en funciones hash sin estado.
Shor	Algoritmo cuántico que resuelve factorización y logaritmo discreto en tiempo polinomial.
HNDL	Harvest Now, Decrypt Later. Capturar tráfico ahora para descifrarlo en el futuro.

Fuente base: presentación de clase POST_QUANTUM_CRYPTOGRAPHY.pdf enviada por el usuario.